DESIGN WALKTHROUGH · OCTOBER 2026

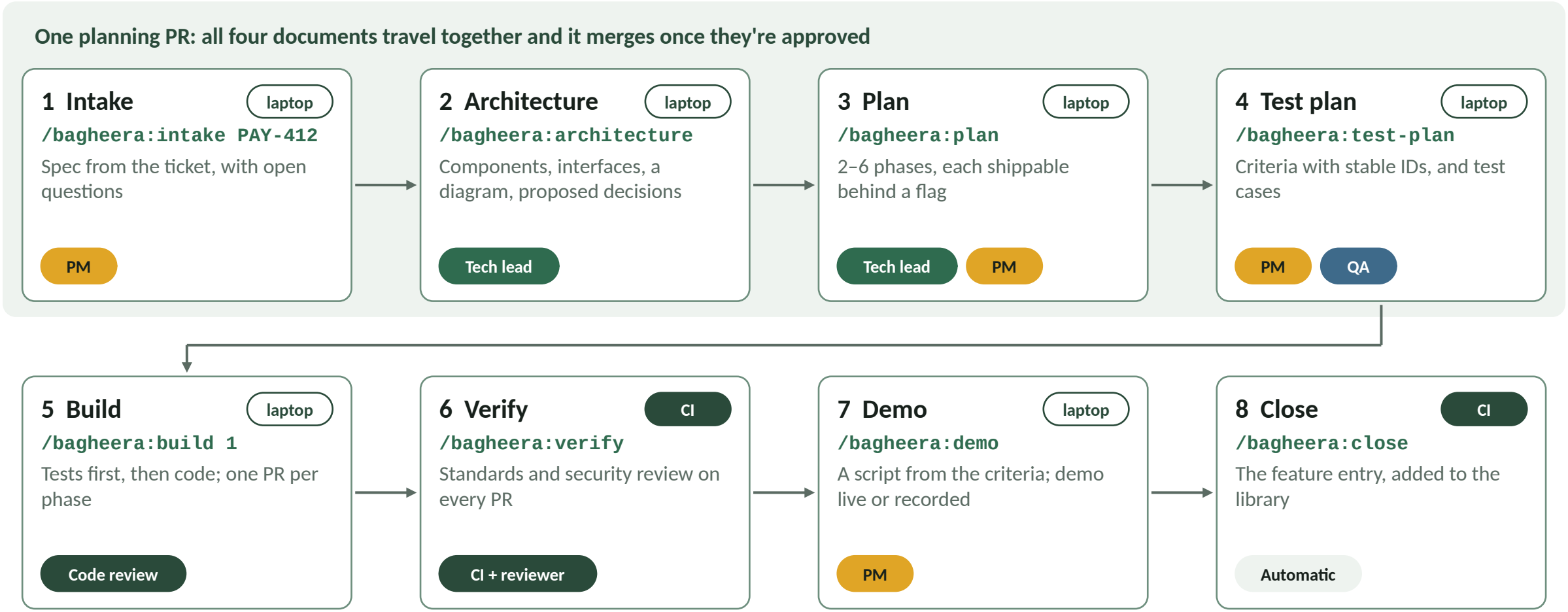
Developer workflow

From a Jira ticket to a shipped feature, step by step

Every command starts with /bagheera. Type /bagheera at any time to see the next step.



The path at a glance



One command per stage

```
/bagheera                  where it stands, what's next
/bagheera:intake PAY-412   1 spec from the ticket
/bagheera:architecture     2 architecture
/bagheera:plan             3 phases
/bagheera:test-plan        4 criteria and test cases
/bagheera:build 2          5 build phase 2
/bagheera:verify           6 runs in CI; preview here
/bagheera:demo             7 writes the demo script
/bagheera:close            8 runs in CI
/bagheera:revise "..."     another iteration
```

Always the full name

A bare /plan opens Claude Code's planning mode. Docs and output always say /bagheera:<stage>.

Type the ticket once

Later stages read it from the branch name, PAY-412/...

Stages start only when asked

Claude can't start a stage by itself. You or a CI workflow do.

Re-run with the same command

Each stage reads only the earlier documents and the library.

The names match the PRD's stages, so commands, documents, the dashboard and the guide use one set of words.

One ticket, one repo, one folder

```
specs/PAY-412/
  spec.md            Intake
  architecture.md    Architecture
  contracts/         only for API files
  plan.md            Plan
  test-plan.md       Test plan
  phase-1-notes.md   Build, one per phase
```

One document per stage

Spec Kit's research and data-model files fold into architecture.md, so approvers read one page.

Readable on the PR page

Diagrams are Mermaid, which GitHub draws for PMs without any tools.

Nothing else in the repo

No index or status file. The demo leaves nothing here.

A frontend and a backend change are two tickets



linked in Jira; each spec lists the other

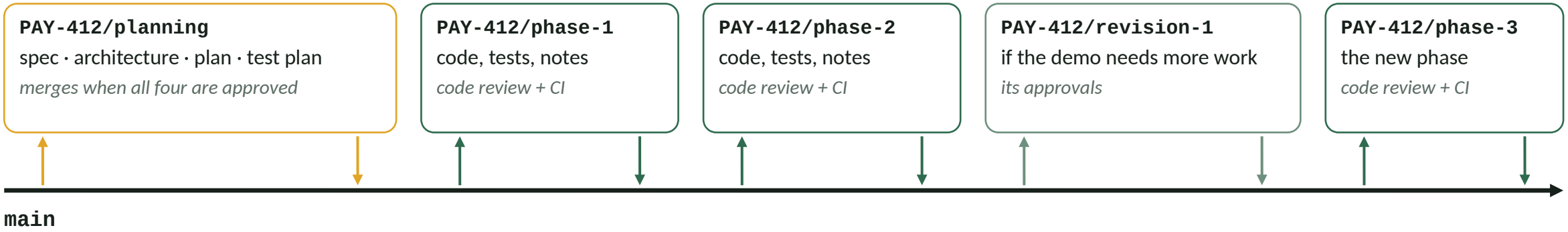
A shared interface is decided once

In PAY-412's architecture. It reaches PAY-413 as a decision in the library; until then it's an open question there.

The harness holds to the rule

- Intake stops if the ticket already has a planning PR in another repo
- CI fails a ticket branch in a repo without that ticket's folder
- Intake asks the PM when a ticket seems to span repos

Branches and PRs that never conflict



1

No two open PRs write the same file

The planning PR writes the folder; each phase only its own notes file. No shared index or status file.

2

Branch from main, never from an open branch

A phase that needs the last one waits for it to merge. No stacked PRs, so no rebases.

3

Update by merging

GitHub's "Update branch". Never rebase or force-push, so anyone's checkout stays valid.

4

Name branches after the ticket

PAY-412/..., never a person, so anyone can pick the work up.

A CI check enforces the file rules. Two features changing the same code can still conflict; that's ordinary engineering.

Open questions stay on the PR page

1 Draft

/bagheera:intake writes the spec with numbered questions and posts them as one PR comment.

2 Discuss on the PR

The PM answers in ordinary comments, inline or not. You ask follow-ups the same way. No commands.

3 Finalise

Run /bagheera:intake again. It writes every answer in, with who gave it and a link.

4 Approve

The PM comments /approved spec, which also confirms the answers were captured right.

On the PR page

Bagheera @payments-pms Q1: Do partial refunds count? Q2: Cards only?

Asha (PM) Q1 yes. Q2 card only for now.

Dev Should wallets come later?

Asha (PM) Yes, as a separate ticket.

```
What lands in spec.md
Q1 Do partial refunds count?
   Yes. @asha, PR comment
Q2 Cards only?
   Card only for now; wallets
   in a new ticket. @asha
```

Only the right role settles a question · numbers run across the feature (Q1, Q2…) · later stages ask the same way

Approval and the stage check

Who approves what

`spec.md`	PM	
`architecture.md`	Tech lead	
`plan.md`	Tech lead	PM
`test-plan.md`	PM	QA
`the demo`	PM	

PM for the criteria, QA for the test cases. The demo is approved on the planning PR.

```
/approved spec   a comment, or in a review

$ /bagheera:architecture
Architecture can't start yet:
spec.md isn't approved.
Waiting on: @seeonee/payments-pms
To go ahead anyway:
/bagheera:architecture --override "<reason>"
```

An approval covers the document as it was
Edit it, and it needs approving again.

Nobody approves their own work
An approval from anyone who changed the document doesn't count.

Overrides are on the record
Posted on the PR and shown on the dashboard. An override isn't an approval.

Build: tests first, one PR per phase

1. **Check**
   All four planning documents approved, and the planning PR merged

2. **Branch**
   PAY-412/phase-2, from main

3. **Tests first**
   Tests named after the phase's criteria; run, fail, committed on their own

4. **Code**
   Until the tests pass, behind the plan's flag

5. **Notes**
   Lessons and gotchas into phase-2-notes.md

6. **PR**
   Pushed and opened, listing the phase's criteria

You steer throughout. Build never changes the test plan: a wrong criterion means a revision.

One ID, from the test plan to the PR

```
test plan     PAY-412-AC-3
              Refunds are rejected after 30 days
test name     test_PAY_412_AC_3_after_30_days
PR comment    PAY-412-AC-3  Pass
```

CI on every phase PR

Criteria coverage	Every criterion in the phase has a test with its ID	fails
Tests first	The PR's first commit is the tests	warns
Results	One PR comment, a row per criterion: pass or fail	shows

Needs each repo's CI to save JUnit test results: one setting at onboarding.

Verify: a second pair of eyes on every PR

Standards and security

Reads the change next to the standards for its language, service and environment. Blocking rules fail the check.

a separate Claude

Matches the plan

The code does what this phase says, and stays inside it.

the same reviewer

Feature folder

The PR writes only its own ticket's files.

a script

Criteria coverage

Every criterion in the phase has a test with its ID.

a script

What it doesn't do

- Run the tests: the repo's own CI still does
- Approve or merge: a person still does
- Stop dangerous commands on laptops: deny rules do (design doc 4)

From your side

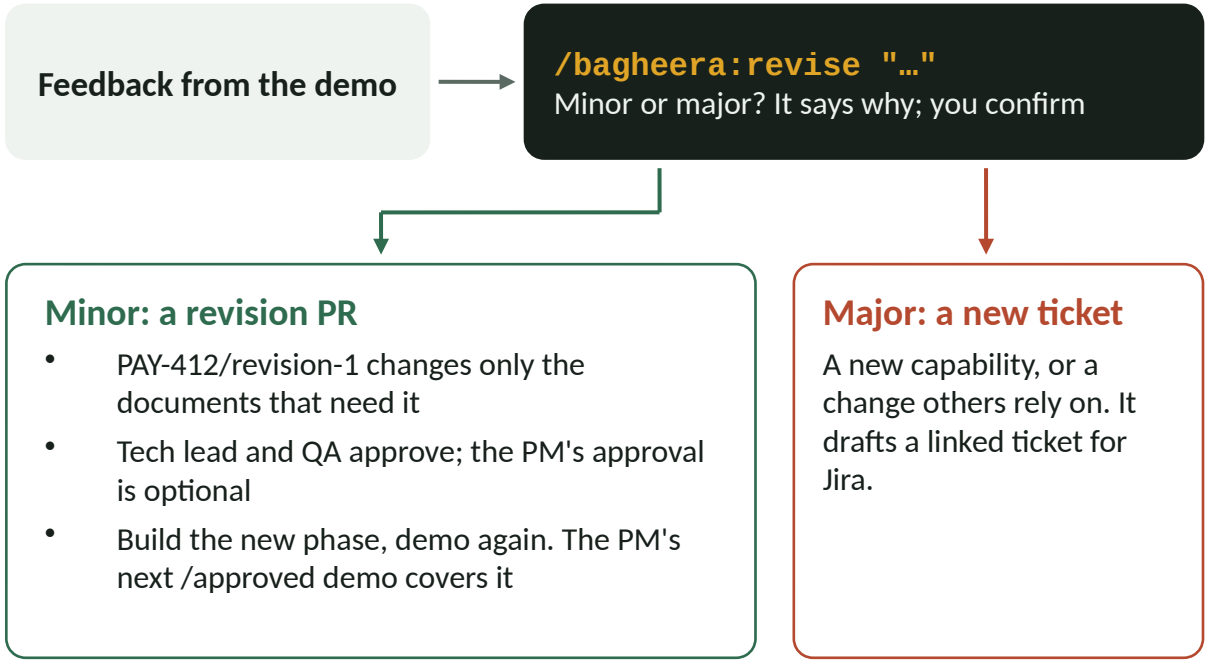
- Push; findings and a check arrive within minutes
- Fix the blocking ones, then a person reviews
- Preview first with /bagheera:verify on your laptop
- Runs on every PR, Bagheera or not

Demo, and another iteration if needed

The demo, off the repo

1. **/bagheera:demo writes the script**
   Per criterion: what to do on stage, with which data, and what the PM should see
2. **You give the demo**
   Live in a meeting, or recorded and shared your way. Nothing is pushed.
3. **The PM accepts**
   /approved demo on the planning PR. Close waits for this.

Another iteration



Numbering carries on: phase 3, PAY-412-AC-9, Q7. The feature stays open until the PM approves a demo.

Handing a ticket over

1

Reassign it in Jira

Nothing to do in Bagheera

2

`/bagheera PAY-412`

Finds the open branch, switches to it, and says where the feature stands and the next command

3

Carry on

Re-run the stage you're in, or keep editing on the same branch

Why it works: nothing lives only on a laptop

Every command pushes

Each stage ends by committing and pushing to the PR

Approvals are on the PR

They belong to the document, not to who wrote it

Answers are in the spec

Questions and answers are on the PR and written in

No chat history needed

Each stage reads only earlier documents and the library

/bagheera warns if the feature folder has changes that were never pushed.

What runs where

On your laptop

- /bagheera
- Intake, Architecture, Plan, Test plan
- Build
- Demo and revise
- Verify, as a preview

With your own GitHub and Jira access, in Claude Code with the plugin

In CI

Verify	**On every PR**	One org ruleset, on repos marked bagheera: on
Criteria results	**Every 15 minutes**	The dashboard sync reads each phase PR's test results
Close	**Every hour**	The library run, once /approved demo is in and every phase has merged
Tests	**As today**	The repo's own CI

Setup GitHub org owners, once: a repo property and one ruleset. Onboarding a repo: the platform team sets bagheera: on.

Moving parts, and who else we need

2 repos	**1** plugin	**3** workflows
1 GitHub App	**1** GitHub Project	**0** servers

Run by the platform team. Nothing self-hosted.

GitHub org owners	App, repo property, one ruleset, role teams	once
Claude admin	Plugin on laptops; CI credentials, spend limit	once
Jira admin	Maybe allow Atlassian's connector for Intake	once
Each team	JUnit test results; review its seeded service pages	onboarding
Approvers	Approve documents and review library PRs	ongoing

Removed along the way

DeepWiki and Docker · a PR per planning stage · CI access to stage and recording storage · per-repo workflow files

Questions to refine together

1. **Minor or major**
   Where's the line for a revision, and who has the final say?
2. **QA in minor revisions**
   Required, as now, or optional like the PM?
3. **Cancelling approvals**
   Should fixing a typo cancel an approval?
4. **Phases in order**
   Should independent phases be built in parallel?
5. **Tests first**
   Is a warning enough, or should a missing tests-first commit fail?
6. **JUnit at onboarding**
   Can every team's CI save JUnit results today?
7. **Org ruleset**
   Who on the GitHub admin side sets it up, and when?
8. **Jira access**
   Does Intake need the Jira admin to allow Atlassian's connector?

Next in the design: standards and the PR review (design doc 4).